

ENTERPRISE GenAI / AI ARCHITECT

INTERVIEW PREPARATION GUIDE

Hard-Level · Scenario-Based · Strategy-Based · 12 Questions with Full Answers & Reasoning

4

Sections

12

Questions

4

Domains

100%

Enterprise Focus

#	Section	Questions
01	Enterprise LLM Architecture & Design	Q1 – Q3
02	RAG, Knowledge Systems & Agents	Q4 – Q6
03	Governance, Risk & Responsible AI	Q7 – Q9
04	Strategy, Leadership & Stakeholder Mgmt	Q10 – Q12

HOW TO USE THIS GUIDE

- Read the question and formulate your own answer BEFORE reading the provided answer.
- The ANSWER section presents the structured response expected at a Principal/Staff level.
- The REASONING section explains WHY each element matters — use this to deepen understanding.
- Tailor answers to your specific industry context and experience during actual interviews.

SECTION 1 — ENTERPRISE LLM ARCHITECTURE & DESIGN

Your organization wants to deploy a multi-tenant GenAI platform serving 50+ business units, each with different data classification levels (Public, Internal, Confidential, Restricted). How do you architect a secure, scalable LLM serving layer?

◆ ANSWER

The architecture must enforce strict data isolation at every layer. The recommended pattern is a centralized AI Gateway with per-tenant context isolation:

- **Tenant Identity Plane:** Each request carries a JWT enriched with tenant-id, data-classification, and allowed-model tiers. The AI Gateway validates this before any LLM call.
- **Model Tiering:** Maintain separate model endpoints (or fine-tuned adapters) per classification tier. Restricted tenants never share inference workers with lower-tier tenants.
- **Prompt/Response Filter Pipeline:** Insert a bi-directional content classifier — input filters block data exfiltration attempts; output filters strip any residual PII or confidential references before returning responses.
- **Vector Store Partitioning:** Each tenant's RAG index lives in a namespaced partition with row-level security (RLS) enforced at query time. Cross-tenant vector contamination is architecturally impossible.
- **Audit & Observability:** Every token exchange is logged to an immutable audit store (append-only S3 + Glue Catalog, or Azure ADLS + Purview) tagged with tenant-id, classification, user, and timestamp.
- **Cost Attribution:** Token metering per tenant feeds a shared showback/chargeback ledger to prevent one BU from starving others.

■ REASONING

Enterprise multi-tenancy is fundamentally a trust-boundary problem. Without tenant isolation at the model-serving layer, a prompt injection from one BU could leak context from another. Data classification drives model selection — Restricted data must never traverse a shared inference worker because inference caches (KV-cache) in high-throughput LLM servers can leak cross-request context. The gateway pattern externalizes cross-cutting concerns (auth, filtering, metering) away from individual application teams, enabling governance without slowing innovation. Interviewers probe whether the candidate defaults to naive shared-endpoint approaches vs. mature zero-trust segmentation.

A Fortune 500 client's customer-facing GenAI chatbot is hitting p99 latency of 8 seconds during peak. SLA requires $p99 < 2s$. Walk through your systematic diagnosis and remediation approach.

◆ ANSWER

Diagnosis in layers:

- **Infrastructure:** Check GPU utilization, memory pressure, and queue depth on the inference cluster. High queue depth = under-provisioning; low GPU utilization = model/batching misconfiguration.
- **Batching Strategy:** Most LLM servers (vLLM, TGI, TensorRT-LLM) support continuous batching. Confirm it is enabled and tuned for the token-length distribution of real traffic.
- **Token Budget:** Audit average prompt + completion length. Long system prompts repeated on every call balloon TTFT (Time To First Token). Move static system prompt content to a prefix cache.
- **KV-Cache Hit Rate:** A low KV-cache hit rate means the server recomputes prefill for every request. Prefix caching for shared system prompts can cut TTFT by 40-70%.
- **Network:** Validate that the client is streaming (SSE/chunked transfer). Even if model latency is 6s, streaming yields first token in <500ms — perceived latency drops dramatically.

Remediation playbook: (1) Enable streaming client-side, (2) Implement prefix caching for system prompts, (3) Right-size batch size and max-concurrent-requests, (4) Horizontal scale inference replicas behind an LB with least-connections routing, (5) Add a semantic cache (e.g., GPTCache, Redis + embedding similarity) for repeat queries, (6) If still insufficient, quantize model (FP16 → INT8/AWQ) for 1.5-2× throughput gain at marginal quality cost.

■ REASONING

Latency problems in LLM systems are multi-factorial and differ fundamentally from traditional web services. Candidates who jump straight to 'scale horizontally' miss that the bottleneck is often the prompt architecture (long prompts, no prefix caching) rather than compute. Streaming is the highest-ROI fix because it changes perceived UX without touching infrastructure. Quantization trades quality for speed and should be validated via regression testing before production rollout. The diagnostic approach — instrumentation before remediation — signals engineering maturity.

The business wants to use GPT-4 for everything. How do you build a business case and technical framework for a model routing strategy that optimizes cost, quality, and latency across a portfolio of use cases?

◆ ANSWER

Model routing is a portfolio optimization problem. The framework has three layers:

- **Task Classification:** Classify each use case on two axes — complexity (reasoning depth required) and risk (accuracy of output matters how much). A simple FAQ lookup is low-complexity/low-risk; contract clause analysis is high-complexity/high-risk.
- **Model Tiers:** Map complexity/risk to model tiers: Tier 1 (nano/small: Haiku, GPT-4o-mini) for retrieval augmentation, summarization, classification. Tier 2 (mid: Sonnet, GPT-4o) for drafting, moderate reasoning. Tier 3 (frontier: Opus, GPT-4, o1) for complex legal/financial reasoning, agentic tasks.
- **Dynamic Router:** Deploy a lightweight router model (fine-tuned BERT or small LLM) that classifies incoming queries and dispatches to the appropriate tier. Confidence thresholds trigger escalation to a higher tier.

Business case: A typical enterprise routes ~60% of queries to Tier 1, 30% to Tier 2, and 10% to Tier 3. If Tier 1 costs 1/20th of Tier 3, average cost drops 70-80% with minimal quality degradation on routed tasks. Validate with A/B testing and LLM-as-judge automated evaluation.

■ REASONING

Using frontier models for every task is the equivalent of routing all network traffic through a satellite link — expensive, slow, and wasteful for simple payloads. The insight that impresses interviewers is framing this as portfolio optimization with explicit risk stratification, not just 'use a cheaper model for easy stuff.' The dynamic router is critical: hard-coding routing logic becomes brittle as use cases evolve. LLM-as-judge automated evaluation enables continuous quality monitoring without human labeling at scale.

SECTION 2 — RAG, KNOWLEDGE SYSTEMS & AGENTS

Q4

RAG Architecture

Design an enterprise RAG system for a global law firm with 2 million legal documents in 12 languages, strict privilege constraints, and a requirement for citation-level accuracy. What are the key architectural decisions?

◆ ANSWER

Key architectural decisions:

- **Privilege-Aware Indexing:** Documents are tagged at ingest with privilege metadata (attorney-client, work-product, public). The retrieval layer enforces privilege checks pre-query — only retrieving documents the querying user is authorized to see. This is non-negotiable for legal.
- **Multilingual Embedding:** Use a multilingual embedding model (e.g., multilingual-e5-large or BGE-M3) that produces a unified embedding space across all 12 languages. Avoid language-specific indexes — cross-language retrieval is required for international matters.
- **Hybrid Search:** Combine dense retrieval (vector similarity) with sparse retrieval (BM25/keyword). Legal text is terminology-heavy; dense-only retrieval misses exact statute citations and case numbers. Reciprocal Rank Fusion (RRF) combines the result lists.
- **Chunk Strategy:** Legal documents have hierarchical structure (section → subsection → clause). Use hierarchical chunking with parent-document retrieval: retrieve at clause level, but return the full section for context. Store chunk-to-document provenance for citation.
- **Citation Verification:** After generation, run a citation grounding verifier — extract all citations from the LLM output and verify each against the retrieved source chunks using entailment scoring. Flag unverified citations for human review.
- **Re-ranking:** Deploy a cross-encoder re-ranker (e.g., Cohere Rerank, BGE re-ranker) on the top-50 retrieved chunks before passing top-10 to the LLM. Improves precision significantly for long-tail legal queries.

■ REASONING

Legal RAG has two hard constraints that generic RAG ignores: privilege and citation accuracy. A hallucinated case citation in a legal memo is a professional liability issue. Privilege-aware retrieval must be enforced at the data layer, not the application layer — otherwise privilege can be circumvented by prompt injection. The hybrid search choice is specific to the domain: legal text relies on exact terminology (statutes, case numbers, defined terms) that dense retrieval underperforms on. Interviewers look for domain-specific reasoning, not generic RAG boilerplate.

You are architecting an autonomous AI agent that will interact with enterprise ERP, CRM, and ITSM systems on behalf of employees. What are the critical safety, reliability, and governance design patterns?

◆ ANSWER

Safety and reliability patterns for enterprise agentic AI:

- **Minimal Privilege Principle:** The agent's API credentials have the narrowest possible scope. An ITSM agent can create tickets but cannot close or delete them without human approval.
- **Action Classification:** Categorize actions as Reversible (read, draft) vs. Irreversible (submit, delete, email). Irreversible actions require a human-in-the-loop confirmation step before execution.
- **Plan-Then-Confirm:** Before executing a multi-step plan, the agent presents its intended action sequence to the user for approval. This prevents runaway automation from a single bad instruction.
- **Guardrail Layer:** Deploy an independent policy-enforcement model that evaluates each proposed action against business rules before execution. This is separate from the reasoning LLM — the fox does not guard the henhouse.
- **Idempotency & Rollback:** All write operations should be idempotent and logged. For critical systems (ERP), implement a staging/dry-run mode — execute in simulation first, show diff, then apply.
- **Circuit Breakers:** If the agent takes N consecutive actions without a successful user confirmation, pause and escalate to a human supervisor. Prevents runaway loops.
- **Full Audit Trail:** Every tool call — input parameters, output, timestamp, user context — is logged immutably. Required for SOX, GDPR, and internal audit.

■ REASONING

Agentic AI in enterprise contexts is where the stakes of failure are highest — a misconfigured agent could submit thousands of spurious purchase orders or close thousands of IT tickets. The minimal privilege and plan-then-confirm patterns are non-negotiable starting points. The separation of the guardrail model from the reasoning LLM is a critical insight: you cannot rely on the same model that chose an action to also validate that action safely — this is a conflict of interest architecturally. Circuit breakers are borrowed from distributed systems resilience patterns and apply equally to agentic AI to prevent infinite tool-call loops.

How do you build a rigorous, scalable evaluation framework for a production RAG system when ground-truth labels are expensive to obtain and the use case evolves rapidly?

◆ ANSWER

A production RAG evaluation framework has four tiers:

- Tier 1 — Reference-Free Metrics (automated, zero cost): Faithfulness (does the answer contradict retrieved context?), Answer Relevance (is the answer on-topic?), Context Precision (are retrieved chunks relevant?), Context Recall (are necessary chunks retrieved?). Use RAGAS or custom LLM-as-judge prompts for all four.
- Tier 2 — Golden Dataset (small, curated): Maintain 200-500 expert-validated question-answer-context triples covering critical use cases. Run regression on every deployment. Add 10-20 new examples per sprint from failure analysis.
- Tier 3 — Human Preference Sampling (continuous): Route 1-2% of live traffic to human raters for blind pairwise comparison between the current and candidate system. Use ELO scoring to aggregate preferences over time.
- Tier 4 — Business Metrics (lagging, ground truth): Track downstream outcomes — helpdesk ticket deflection rate, query resolution rate, escalation rate. These are the ultimate arbiter but lag by days/weeks.

Rapid evolution strategy: Use LLM-as-judge with a critiquing prompt rather than fixed rubrics. The judge model evaluates fluency, faithfulness, and completeness against the retrieved context — no ground-truth label needed. Calibrate judge agreement against human ratings quarterly.

■ REASONING

The naive answer is 'use human evaluation.' The sophisticated answer recognizes that human evaluation does not scale to thousands of daily queries and does not provide the fast feedback loop that production systems need. The four-tier framework balances speed (automated metrics), reliability (golden dataset), human signal (preference sampling), and business impact (business metrics). LLM-as-judge is now standard but must be calibrated — an uncalibrated judge is just expensive noise. The key insight for rapidly evolving use cases is building the evaluation infrastructure as a product, not a one-time exercise.

SECTION 3 — GOVERNANCE, RISK & RESPONSIBLE AI

Q7

AI Governance

The EU AI Act classifies certain enterprise AI applications as 'high-risk.' How do you operationalize AI Act compliance for a global enterprise with 80+ AI/ML systems, without creating a compliance bottleneck?

◆ ANSWER

Operationalizing EU AI Act compliance at scale requires a risk-tiered governance model:

- **System Inventory & Classification:** Build an AI system registry — a living catalog of all 80+ systems tagged with risk tier (Prohibited, High-Risk, Limited-Risk, Minimal-Risk) based on EU AI Act Annex III criteria. High-Risk flags include: HR screening, credit scoring, biometric identification, critical infrastructure, law enforcement.
- **Compliance-as-Code:** Embed compliance requirements into CI/CD pipelines as automated gates. High-Risk systems trigger mandatory documentation artifacts: Fundamental Rights Impact Assessment (FRIA), Technical Documentation (Annex IV), Conformity Assessment. These are templated and version-controlled.
- **Federated AI Ethics Board:** Rather than centralizing all review, create a hub-and-spoke model — a central AI Risk Office sets standards; embedded AI Risk Stewards in each BU conduct first-pass reviews. Only systems that exceed risk thresholds escalate centrally.
- **Human Oversight by Design:** High-Risk systems must demonstrate meaningful human oversight. Architect this as a technical requirement: systems must provide confidence scores, explainability artifacts, and override mechanisms — not just a policy statement.
- **Ongoing Monitoring:** Post-deployment, High-Risk systems require quarterly bias audits, drift monitoring, and incident reporting. Integrate this into your MLOps platform (MLflow, SageMaker, Vertex) as automated drift alerts.

■ REASONING

The failure mode of compliance programs is creating heavyweight processes that all 80 teams must navigate for every system — this produces either paralysis or shadow AI (teams hide systems to avoid compliance). The hub-and-spoke model and risk tiering solve this: 70% of systems are Minimal-Risk and need only basic documentation; 10% are High-Risk and receive intensive scrutiny. Compliance-as-code is critical for scale — manual checklists do not survive contact with a 6-week sprint cadence. Interviewers probe whether candidates understand that EU AI Act compliance is an engineering problem as much as a legal one.

Your enterprise LLM application has been identified as a target for prompt injection attacks, jailbreaks, and data exfiltration via the model. Design a layered defense strategy.

◆ ANSWER

Defense in depth for LLM applications — five layers:

- Layer 1 — Input Sanitization: Classify incoming prompts with a dedicated safety classifier (trained or API-based) before they reach the main LLM. Flag and block known injection patterns, role-play jailbreak structures, and encoded payloads (base64, rot13).
- Layer 2 — Privilege-Separated Architecture: The LLM operates with a read-only system context. Tool calls (database queries, API calls) are executed by a separate, narrowly privileged executor — the LLM cannot directly access credentials or raw data stores.
- Layer 3 — Output Filtering: Every LLM response passes through an output classifier before delivery. Detect and redact PII, secrets (API keys, passwords via regex + ML), and confidential pattern matches before the response leaves the system.
- Layer 4 — Canary Tokens: Inject synthetic confidential strings (canary tokens) into the system prompt and knowledge base. If a canary token appears in output, it is a confirmed data exfiltration event — trigger immediate incident response.
- Layer 5 — Behavioral Monitoring: Maintain a baseline of normal interaction patterns (query length, tool call frequency, data access patterns). Flag statistical anomalies — an unusually long query repeatedly hitting sensitive data segments is a red flag.

Indirect prompt injection: Harden against injections embedded in retrieved documents (RAG). Treat all retrieved content as untrusted third-party input — apply the same input classifier to retrieved chunks before inserting them into the prompt.

■ REASONING

Prompt injection is the SQL injection of the LLM era — it exploits the fundamental property that instructions and data share the same channel in natural language. No single control eliminates it; defense must be layered. Canary tokens are a practitioner insight that interviewers rarely hear — they turn a detection problem (did exfiltration occur?) into a near-certain signal. Indirect prompt injection via RAG is the most underestimated attack vector in enterprise deployments — most teams secure the user input but ignore that injected instructions could arrive through a retrieved web page or document.

An AI hiring screening tool your team built is showing disparate impact against candidates from certain demographic groups, discovered 6 months after deployment. Walk through your incident response and long-term remediation.

◆ ANSWER

Immediate incident response (first 72 hours):

- Suspend automated screening decisions. Switch to human-in-the-loop mode immediately. Do not wait for root cause to protect candidates.
- Scope assessment: Quantify affected population — how many candidates were screened, what is the demographic breakdown, what decisions were made that cannot be reversed (rejections sent)?
- Legal notification: Engage legal counsel — depending on jurisdiction (EEOC in US, GDPR in EU, CCPA in California), there may be mandatory notification obligations.

Root cause analysis:

- Training data audit: Examine training data for historical bias (e.g., trained on past hires that reflected historical discrimination). Historical hiring data almost always encodes past bias.
- Feature analysis: Identify which features drive adverse impact. Proxies for protected characteristics (zip code, school name, name) are common hidden discriminators.

Remediation:

- Rebalance training data using re-weighting, oversampling of underrepresented groups, or synthetic data augmentation.
- Apply fairness constraints during training (equalized odds, demographic parity) and validate with disaggregated metrics by demographic group.
- Implement a fairness monitoring dashboard — ongoing disaggregated performance tracking — as a permanent production gate.
- Consider third-party bias audit before re-deployment (as required in New York Local Law 144).

■ REASONING

The suspension decision is non-negotiable and must come first — before root cause, before communications. Continuing to make automated decisions with a known-biased system creates compounding legal and reputational liability. The historical data insight is critical: models trained on past hiring decisions inherit the biases of past hiring managers. Interviewers probe depth of understanding on fairness metrics — equalized odds (equal true positive and false positive rates across groups) is often the right constraint for hiring; demographic parity (equal selection rates) is appropriate when base rates should be equalized. Candidates must demonstrate they understand the difference.

SECTION 4 — STRATEGY, LEADERSHIP & STAKEHOLDER MANAGEMENT

The CEO asks you to deliver a '10× productivity' improvement via GenAI within 12 months. How do you translate this mandate into a credible AI strategy with measurable outcomes?

◆ ANSWER

The first move is to reframe the mandate before accepting it as stated:

- Decompose '10×': 10× productivity is not a single metric — it is a composite across different work types. Start with a time-motion study across 3-5 high-volume knowledge worker roles. Quantify where time is spent (e.g., drafting: 4h/week; research: 6h/week; status reporting: 3h/week).
- Set use-case-specific targets: If drafting takes 4h/week and GenAI reduces it to 30 minutes, that is an 8× improvement on that task. Aggregate across tasks to build a credible portfolio productivity model.
- Portfolio approach — three horizons: Horizon 1 (0-3 months): Quick wins with proven ROI — copilots for code, document drafting, meeting summarization. Horizon 2 (3-9 months): Process automation — automated report generation, RAG-powered knowledge retrieval. Horizon 3 (9-12 months): Workflow transformation — end-to-end agentic workflows replacing multi-step human processes.
- Measurement framework: Define baseline metrics before deployment. Track adoption rate, task completion time, error rate, employee satisfaction (AI is useful vs. AI is reliable). Report monthly to executive sponsor.
- Manage expectations: Communicate that '10×' is achievable on specific high-frequency, low-complexity tasks but unlikely as an enterprise-wide average in 12 months. Frame realistic target: 3-5× average productivity uplift on targeted knowledge work in Year 1.

■ REASONING

Accepting '10×' uncritically is a career risk — if the architect builds a strategy against an impossible benchmark, failure is guaranteed. The reframing move (decompose the metric, set task-specific targets, build a portfolio) demonstrates strategic maturity. The three-horizons model ensures early wins maintain executive confidence while longer-term transformational work progresses. The explicit expectation management — telling the CEO that 10× enterprise-wide is not achievable in 12 months — is the right call even if uncomfortable. Interviewers look for candidates who lead from data, not from compliance with whoever is loudest.

The CTO wants to build a proprietary foundational LLM. The Chief Procurement Officer wants to buy/license from an existing provider. How do you structure and lead this decision for a 15,000-person enterprise?

◆ ANSWER

Structure the decision as a rigorous make-vs-buy analysis across five dimensions:

- **Differentiation:** Does a proprietary model create durable competitive advantage? For 99% of enterprises, the answer is no — the moat is in proprietary data, workflows, and distribution, not in the base model weights.
- **Cost reality:** Training a frontier-class LLM requires \$50-500M+ in compute, 100+ ML researchers, and 18-36 months. This is the budget of an AI lab, not a typical enterprise R&D; allocation. Fine-tuning or RAG on top of existing models delivers 80% of the value at 1% of the cost.
- **Time-to-value:** API access to GPT-4/Claude/Gemini enables production deployment in weeks. Custom pre-training delays value realization by 18+ months — a lifetime in the current competitive environment.
- **Data moat:** The compelling middle path — use a best-in-class base model via API, fine-tune on proprietary data (domain adaptation), and build RAG pipelines on proprietary knowledge. The competitive advantage is the data and the application, not the weights.
- **Risk:** A single-provider API dependency is a supply-chain risk. Mitigate with a model-agnostic abstraction layer (LiteLLM, AWS Bedrock, Azure AI) enabling provider switching without application rewrites.

Recommendation: Buy (API/managed) for the base model. Fine-tune and RAG for domain specificity. Invest the CTO's budget in proprietary data pipelines, evaluation infrastructure, and AI product development — that is the defensible moat.

■ REASONING

This question tests whether the architect can manage ego-driven technical decisions (building because it is impressive) versus value-driven ones. The data point on training costs is critical — most CTOs do not have accurate mental models for what frontier model training actually costs. The middle path (buy + fine-tune + RAG) is the correct answer for almost every enterprise and demonstrates nuance. The supply-chain risk point and the abstraction layer mitigation show systems thinking beyond the immediate decision.

Six months into a GenAI transformation program, employee adoption is at 12% despite mandatory training. Three senior managers are actively discouraging their teams from using the AI tools. How do you diagnose and recover the program?

◆ ANSWER

Diagnosis — root cause before solutions:

- Adoption data segmentation: Break 12% by department, seniority, and use case. If adoption is 40% in one BU and 2% in others, the solution differs from uniform underperformance.
- User research: Conduct structured interviews with 20-30 non-adopters. The three most common blockers are: tool does not solve a real pain point I have (fit problem); I do not trust the outputs (reliability problem); I am afraid it will replace me (psychological safety problem).
- Manager behavior root cause: Understand why three managers are discouraging use. Is it: (a) they tried it and it produced poor output for their team's specific work, (b) they are protecting headcount, or (c) they feel their expertise is being devalued?

Recovery strategy:

- Address the managers directly: Bring them into the program design — make them AI Champions rather than resisters. Give them early access to new features, invite them to evaluation sessions. Resistance often becomes advocacy when people feel ownership.
- Pivot from tool-push to problem-pull: Stop promoting 'AI tools.' Start with 'what takes you the most time each week?' Then build targeted AI solutions for that specific pain. Adoption follows perceived value, not mandates.
- Psychological safety: Communicate explicitly from executive leadership that AI is a productivity amplifier, not a headcount reduction mechanism. Back this with a public commitment: no AI-driven layoffs in the next 24 months.
- Success stories: Identify the 12% who are adopting and surface their stories — peer testimonials are 10× more effective than training.

■ REASONING

Low adoption is almost never solved by more training — it signals a product-market fit problem or a trust problem, both of which require qualitative diagnosis. The impulse to escalate past resistant managers is counterproductive; converting them to champions is far more powerful. The psychological safety point is critical and often ignored by technology-focused architects — employees rationally resist tools they believe threaten their employment. An explicit no-layoffs commitment from leadership, backed by actual behavior, is the single highest-leverage intervention for adoption barriers in GenAI programs.